Experiment No.3
Design & Implementation of Two Pass Macro Processor
Date of Performance:30/03/26
Date of Submission:06/04/26

Aim: Design & Implementation of Pass 1 of Two Pass Macro Processor.

Objective: To study and implement Pass 1 of two pass Macro Processor for IBM 360 Machine.

Theory:

Macro: A macro is a unit of specification for program generation through expansion. A macro instruction is a notational convenience for the programmer. It allows the programmer to write shorthand version of a program (module programming).

Macro Processor: The macro processor replaces each macro invocation with the corresponding sequence of statements.

You may design a two-pass macro processor.

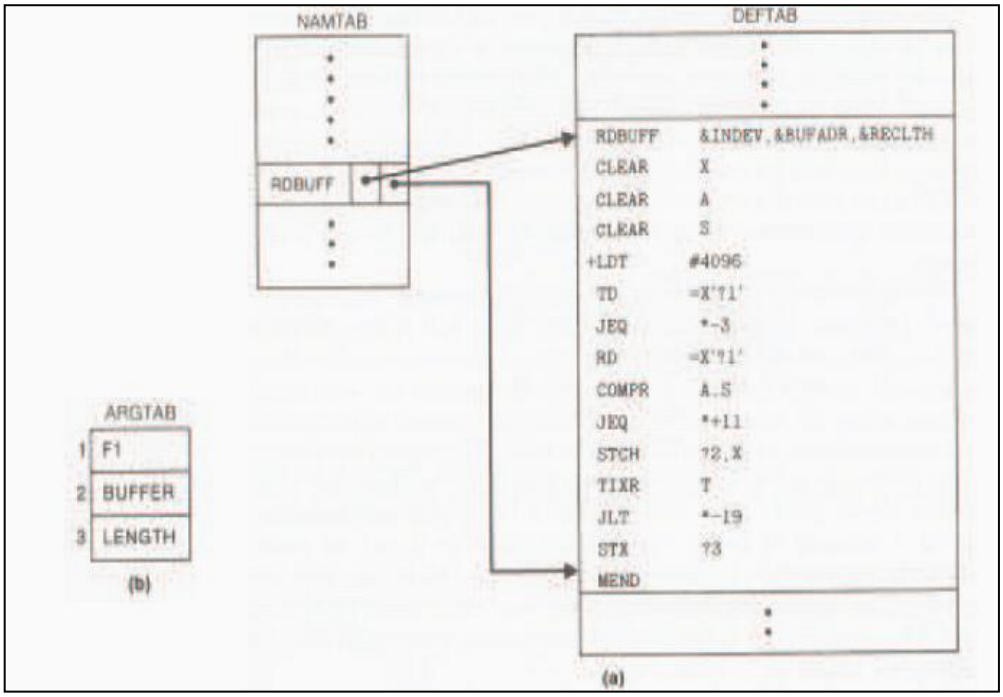
Pass 1: Process all macro definitions.

- Identifies macro definitions and calls in the program
- Determine the formal parameters & their values

Pass 1 Databases of Macro Processor

1. Copy of source program statements
2. Output macro source listings for use by Pass-2
3. MDT to store body of macro definition
4. MDTC used to points the next entry towards in MDT
5. MNT to store names of macro defines in the program
6. MNTC used to points towards next entry into MNT.
7. An array called Parameter List Array (PLA) to manage an index for formal parameters (arguments)

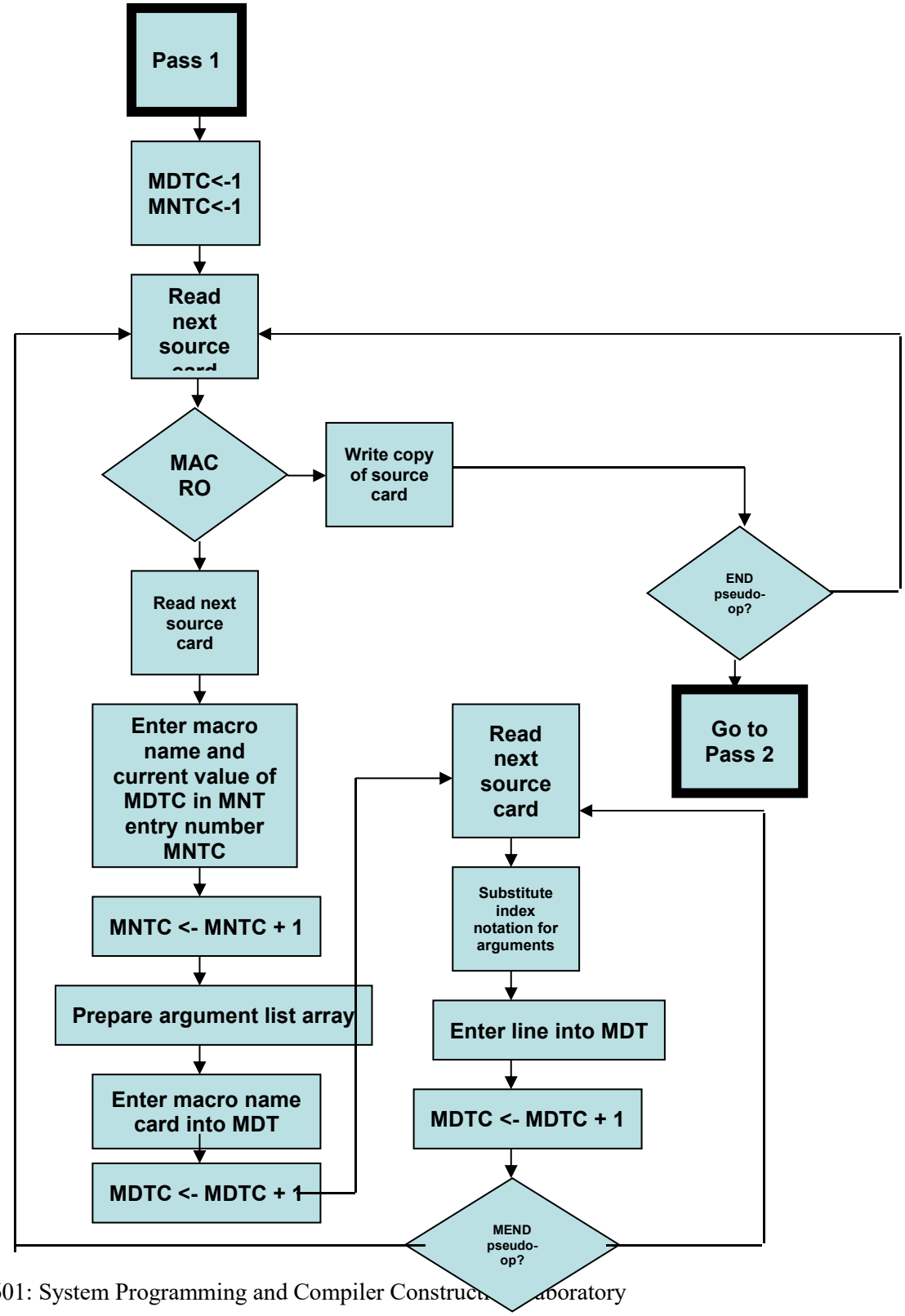
Data structure



PASS-I of Macro Processor- Processing Macro Definitions

1. Initialize MDTC and MNTC
2. Read the next source statement of the program.
3. If the statement contains MACRO pseudo-op. go to step 6.
4. Output the instruction of the statement.
5. If the statement contains END pseudo-op, go to PASS-II else go to step 2
6. Read the next source statement of the program.
7. Make an entry of the macro name and MTDC in MNT at location MNTC and increment MNTC by 1.
8. Prepare the parameter (arguments) list array.
9. Enter macro name into MDT and increment MTDC by 1.
10. Read the next card and substitute index for the parameters (arguments).
11. Enter line into MDT and increment MDT by 1.
12. If MEND pseudo-op is found, go to step 2 else go to step 10

Pass 1 flowchart:



Code:

```
#include <stdio.h>
#include <string.h>

struct MNT
{
    char name[20];
    int mdt_index;
};

struct MDT
{
    char line[80];
};

struct ALA
{
    char param[20];
};

int main()
{
    struct MNT mnt[10];
    struct MDT mdt[100];
    struct ALA ala[10];

    int mntc=0, mdtc=0, alac=0;
    int n,i;
```

```
char program[50][80];
char line[80];
char macroName[20];

printf("Enter number of lines in program: ");
scanf("%d",&n);
getchar();

printf("\nEnter program:\n");

for(i=0;i<n;i++)
   fgets(program[i],80,stdin);

for(i=0;i<n;i++)
{
   strcpy(line,program[i]);
   line[strcspn(line,"\n")]=0;

   if(strcmp(line,"MACRO")==0)
   {
      i++;

      strcpy(line,program[i]);
      line[strcspn(line,"\n")]=0;

      sscanf(line,"%s",macroName);

      strcpy(mnt[mntc].name,macroName);
      mnt[mntc].mdt_index=mdtc;
      mntc++;

      strcpy(mdt[mdtc].line,line);
```

```
        mdtc++;

        char *token=strtok(line," ,");

        token=strtok(NULL," ,");

        while(token!=NULL)
        {
            strcpy(ala[alac].param,token);
            alac++;
            token=strtok(NULL," ,");
        }

        while(1)
        {
            i++;
            strcpy(line,program[i]);
            line[strcspn(line,"\n")]=0;

            strcpy(mdt[mdtc].line,line);
            mdtc++;

            if(strcmp(line,"MEND")==0)
                break;
        }
    }
}

printf("\nMNT (Macro Name Table)\n");
printf("Index\tName\tMDT Index\n");
```

```
    for(i=0;i<mntc;i++)
        printf("%d\t%s\t%d\n",i+1,mnt[i].name,mnt[i].mdt_index);

    printf("\nMDT (Macro Definition Table)\n");

    for(i=0;i<mdtc;i++)
        printf("%d\t%s\n",i+1,mdt[i].line);

    printf("\nALA (Parameter List Array)\n");

    for(i=0;i<alac;i++)
        printf("%d\t%s\n",i+1,ala[i].param);

    return 0;
}
```

Output:

```
Enter number of lines in program: 8

Enter program:
MACRO
INCR &A,&B
LDA &A
ADD &B
STA &A
MEND
START
INCR X,Y
END

MNT (Macro Name Table)
Index   Name    MDT Index
1   INCR    0

MDT (Macro Definition Table)
1   INCR &A,&B
2   LDA &A
3   ADD &B
4   STA &A
5   MEND

ALA (Parameter List Array)
1   &A
2   &B
```

Conclusion:

1. Comment on working of Program by providing input source program.
2. Comment on databases generated during processing of macro definition.

Comment on working of Program by providing input source program.

The program implements Pass-2 of a Two Pass Assembler which generates the target machine code using the intermediate code produced in Pass-1. It reads the symbol table and literal table containing the addresses of symbols and literals. The program then processes the intermediate code instructions such as imperative statements (IS), declarative statements (DL), and assembler directives (AD). For each instruction, it replaces symbolic operands with their corresponding addresses from the tables. Finally, it generates the target code along with the updated location counter. For example, an input intermediate instruction like (IS,04) (R,1) (S,1) will produce

machine code using the address of symbol S1 from the symbol table.

Comment on databases generated during processing of macro definition.

During macro processing, several databases are generated to manage macro definitions and their expansion. The Macro Name Table (MNT) stores the name of each macro along with a pointer to its definition in the macro definition table. The Macro Definition Table (MDT) stores the actual macro instructions and statements. The Argument List Array (ALA) is used to maintain the mapping between formal parameters and actual parameters used during macro calls. These databases help the macro processor efficiently expand macro calls during program execution. As a result, the assembler can replace macro invocations with the corresponding macro body during processing.